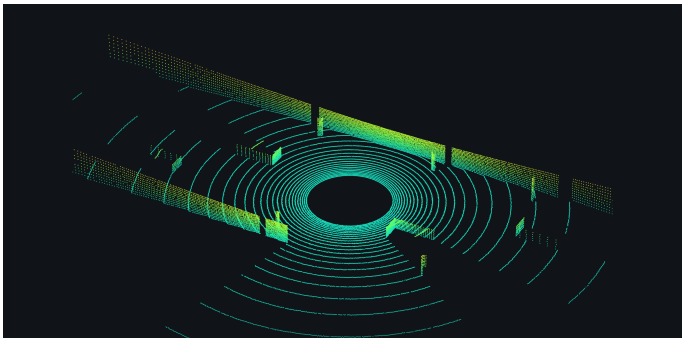
3D Deep Learning

Modern Computer Vision | Spring 2026

Oren Chikli | Tutorial 7

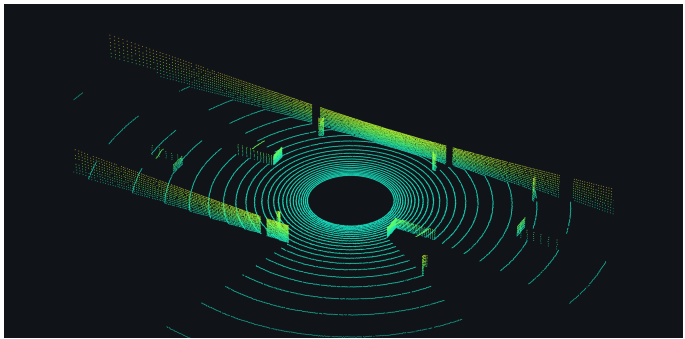
This presentation largely relies on Stanford CS231n: Deep Learning for Computer Vision (Spring 2025), Lecture 15: 3D Vision, Jiajun Wu.

Why 3D?



one sweep of a 32-beam LiDAR (synthetic)

Why 3D?



one sweep of a 32-beam LiDAR (synthetic)

- An image is a **2D projection** of a 3D world
- Robots, AR, autonomous cars must *act* in that world
- Today: how neural nets **represent** and **process** 3D

Today

- **Representing 3D** — how do we even *write a shape down*? (points, meshes, voxels)

Today

- **Representing 3D** — how do we even *write a shape down*? (points, meshes, voxels)
- **Learning on 3D** — how does a neural network *consume* each format? Three ideas, in historical order:
  1. turn the shape into ordinary *photos*
  2. turn it into *3D pixels*
  3. feed the *raw points* to a network built for them

Today

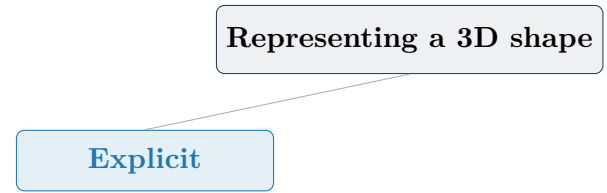
- **Representing 3D** — how do we even *write a shape down*? (points, meshes, voxels)
- **Learning on 3D** — how does a neural network *consume* each format? Three ideas, in historical order:
  1. turn the shape into ordinary *photos*
  2. turn it into *3D pixels*
  3. feed the *raw points* to a network built for them
- **Comparing 3D** — when are two shapes "the same"?

Many Ways to Represent a Shape

Representing a 3D shape

After Ren Ng, CS231n

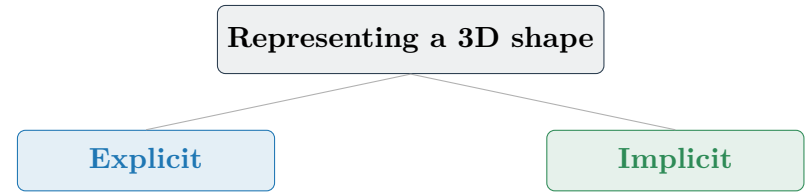
Many Ways to Represent a Shape



- point cloud
- polygon mesh
- voxels
- subdivision, NURBS ...

After Ren Ng, CS231n

Many Ways to Represent a Shape



After Ren Ng, CS231n

Representation Considerations

- Must be **stored** in the computer
- **Creation** of new shapes — input metaphors, interfaces ...
- **Operations** — editing, simplification, smoothing, filtering, repairing ...
- **Rendering** — rasterization, ray tracing, neural rendering ...
- **Animation**

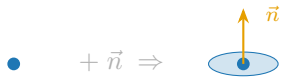
Point Clouds

- Simplest representation: **only points**, no connectivity
- A set of (x, y, z) coordinates, possibly with a normal

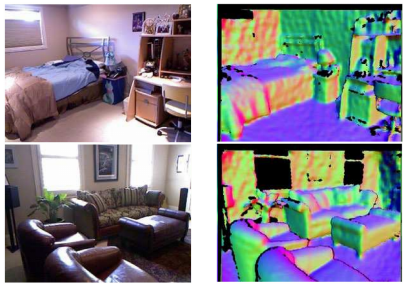
Normals: Eigen & Fergus, ICCV 2015 (via CS231n L18)

Point Clouds

- Simplest representation: **only points**, no connectivity
- A set of (x, y, z) coordinates, possibly with a normal
- Points with orientation are called **surfels**



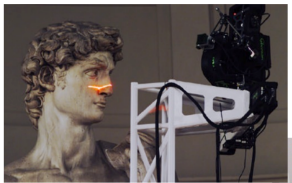
point (x, y, z) surfel: oriented disc

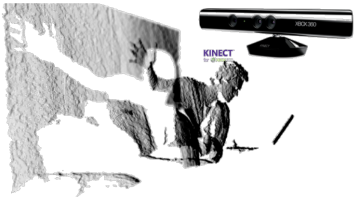


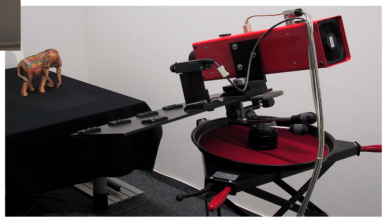
RGB (left) → normals (right; colour = direction)
Normals: Eigen & Fergus, ICCV 2015 (via CS231n L18)

Where Point Clouds Come From

The native output of **3D scanners** — laser, structured light, LiDAR, depth cameras







Figures: Hao Su, CS231n

Where Point Clouds Come From

The native output of **3D scanners** — laser, structured light, LiDAR, depth cameras

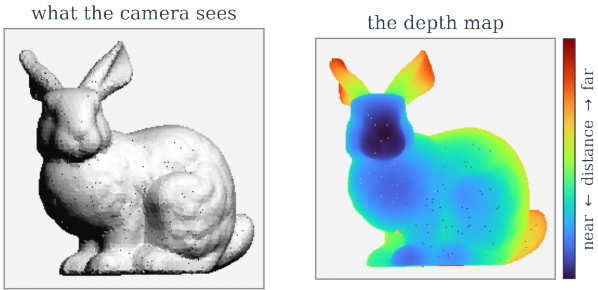
Set of raw scans

potentially noisy — and one scan sees one viewpoint, so multiple partial scans must be **registered** together

Figures: Hao Su, CS231n

Depth Maps (RGB-D)

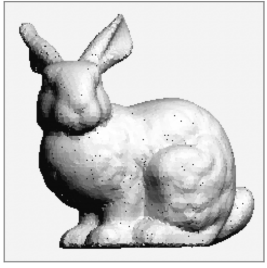
- An **image** where every pixel stores the **distance** to the camera
- Native output of Kinect, iPhone Face ID, stereo rigs



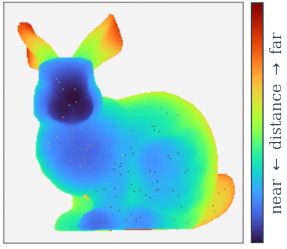
Depth Maps (RGB-D)

- An **image** where every pixel stores the **distance** to the camera
- Native output of Kinect, iPhone Face ID, stereo rigs
- “2.5D”: one viewpoint — the far side is still missing
- For a CNN it is just an image with **one more channel**

what the camera sees



the depth map

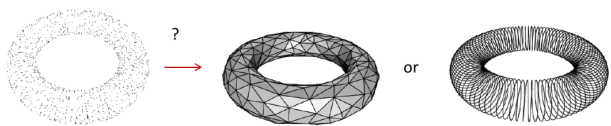


Point Clouds: Strengths and Limits

+ Represent **any** geometry
+ Scale to huge datasets

Point Clouds: Strengths and Limits

+ Represent **any** geometry
+ Scale to huge datasets
– No surface, no **topology**
– Hard to render smoothly in undersampled regions

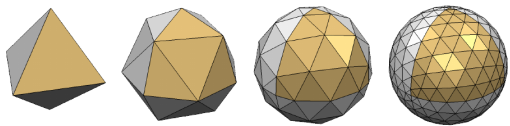


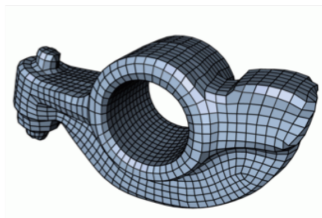
same points — two different surfaces

Figure: Hao Su, CS231n

Polygon Meshes

- **Boundary representation** of objects: vertices + faces (usually triangles)
- Connectivity solves the ambiguity points have
- Triangle count is a **resolution knob**





Figures: Hao Su, CS231n

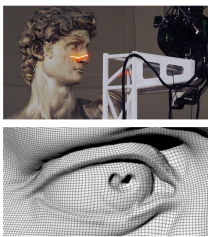
A Large Triangle Mesh

David

Digital Michelangelo Project

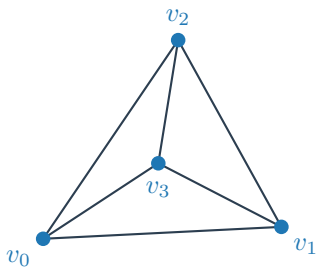
28,184,526 vertices

56,230,343 triangles



Figures: Ren Ng, CS231n

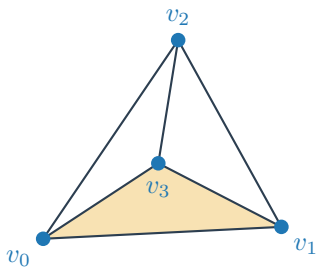
A Mesh Is Two Matrices



vertices V — **floats**, one row per point:

$$V = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0.5 & 0.9 & 0 \\ 0.5 & 0.3 & 0.8 \end{pmatrix} \begin{matrix} \leftarrow v_0 \\ \leftarrow v_1 \\ \leftarrow v_2 \\ \leftarrow v_3 \end{matrix}$$

A Mesh Is Two Matrices



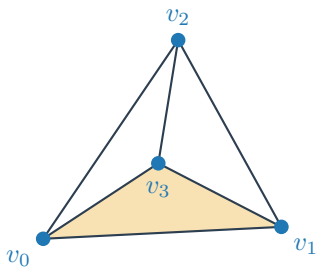
vertices V — **floats**, one row per point:

$$V = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0.5 & 0.9 & 0 \\ 0.5 & 0.3 & 0.8 \end{pmatrix} \begin{matrix} \leftarrow v_0 \\ \leftarrow v_1 \\ \leftarrow v_2 \\ \leftarrow v_3 \end{matrix}$$

faces F — **integers**: three row-*indices* of V per triangle:

$$F = \begin{pmatrix} \mathbf{0} & \mathbf{1} & \mathbf{3} \\ 1 & 2 & 3 \\ 0 & 2 & 3 \\ 0 & 1 & 2 \end{pmatrix}$$

A Mesh Is Two Matrices



vertices V — **floats**, one row per point:

$$V = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0.5 & 0.9 & 0 \\ 0.5 & 0.3 & 0.8 \end{pmatrix} \begin{matrix} \leftarrow v_0 \\ \leftarrow v_1 \\ \leftarrow v_2 \\ \leftarrow v_3 \end{matrix}$$

faces F — **integers**: three row-*indices* of V per triangle:

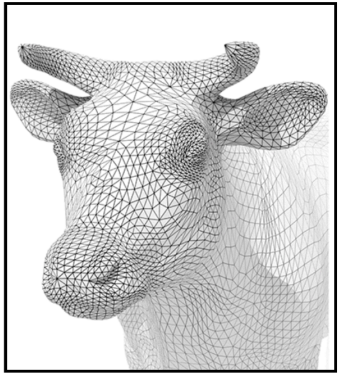
$$F = \begin{pmatrix} \mathbf{0} & \mathbf{1} & \mathbf{3} \\ 1 & 2 & 3 \\ 0 & 2 & 3 \\ 0 & 1 & 2 \end{pmatrix}$$

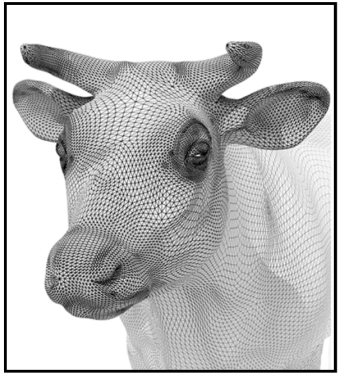
rows of F that **share indices** share an edge — connectivity comes for free, no coordinate is stored twice.

David is the same two matrices: V with 28M rows, F with 56M rows.

Mesh Operations

- **Subdivision** — upsample by interpolation

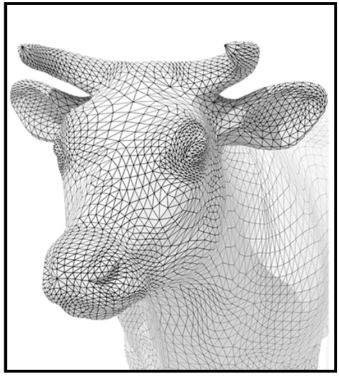


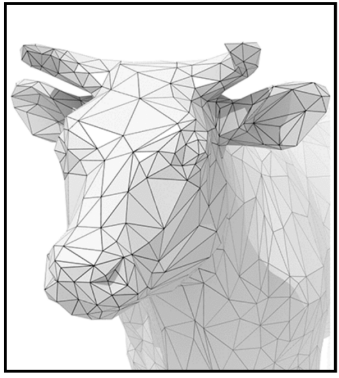


Cow figures: Ren Ng, CS231n

Mesh Operations

- **Subdivision** — upsample by interpolation
- **Simplification** — downsample, keep shape

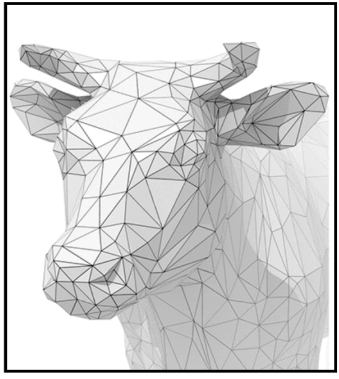


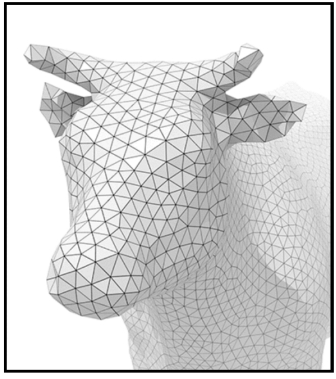


Cow figures: Ren Ng, CS231n

Mesh Operations

- **Subdivision** — upsample by interpolation
- **Simplification** — downsample, keep shape
- **Regularization** — equalize triangle quality

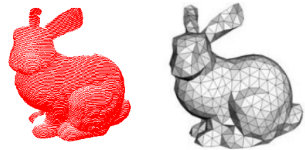




Cow figures: Ren Ng, CS231n

Shape Representations

✓ **Non-parametric** — store samples of the surface directly
points, meshes *(and voxels, coming up)*

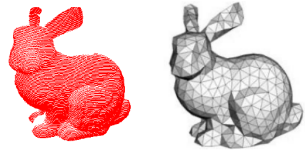


After Hao Su / Ren Ng, CS231n

Shape Representations

✓ **Non-parametric** — store samples of the surface directly
points, meshes *(and voxels, coming up)*

– **Parametric** — a function $s(u, v)$ you *evaluate*
splines, subdivision surfaces — not covered



After Hao Su / Ren Ng, CS231n

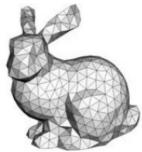
Shape Representations

✓ **Non-parametric** — store samples of the surface directly
points, meshes *(and voxels, coming up)*

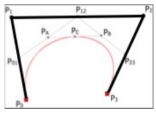
– **Parametric** — a function $s(u, v)$ you *evaluate*
splines, subdivision surfaces — not covered

→ **Implicit** — a function whose *zero-set* is the surface
a quick look, next

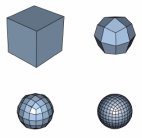
Points



Meshes



Splines

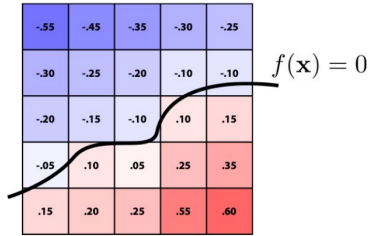


Subdivision Surfaces

After Hao Su / Ren Ng, CS231n

Implicit Surfaces: Level Sets on a Grid

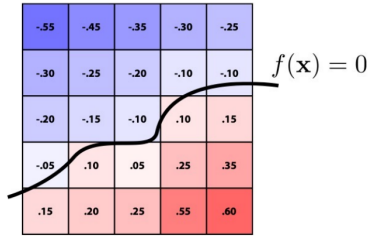
- **Implicit**: define the shape by a function f; the surface is the level set $\{f = 0\}$



After Ren Ng, CS231n

Implicit Surfaces: Level Sets on a Grid

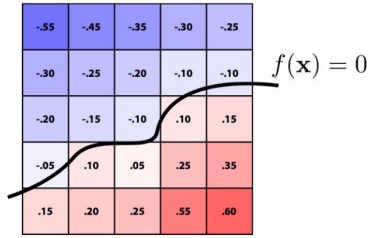
- **Implicit**: define the shape by a function f; the surface is the level set $\{f = 0\}$
- A clean *formula* for f is hopeless for real shapes — so **don't use one**



After Ren Ng, CS231n

Implicit Surfaces: Level Sets on a Grid

- **Implicit**: define the shape by a function f; the surface is the level set $\{f = 0\}$
- A clean *formula* for f is hopeless for real shapes — so **don't use one**
- **Fill a grid with values** instead: a sensor writes them (CT density), or you compute distance-to-surface. Surface = the **zero-crossing**

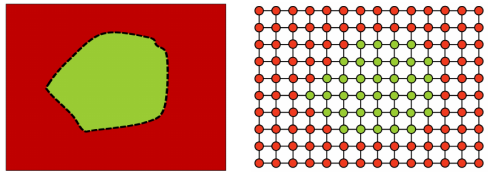


CT: the scanner fills the grid with tissue density

After Ren Ng, CS231n

Voxels — Threshold the Grid

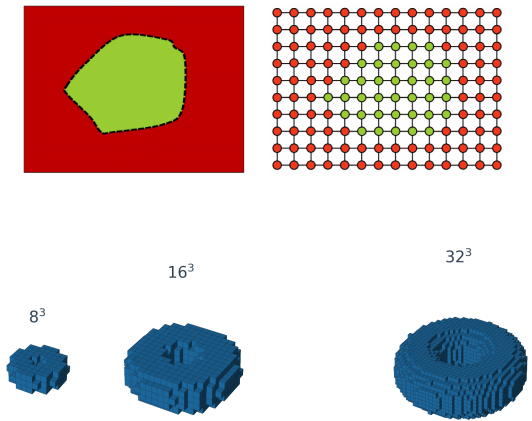
- **Binary-threshold** that grid: 1 inside, 0 outside
- A 3D **occupancy grid** — the direct analog of pixels



Thresholding figure after CS231n; tori generated

Voxels — Threshold the Grid

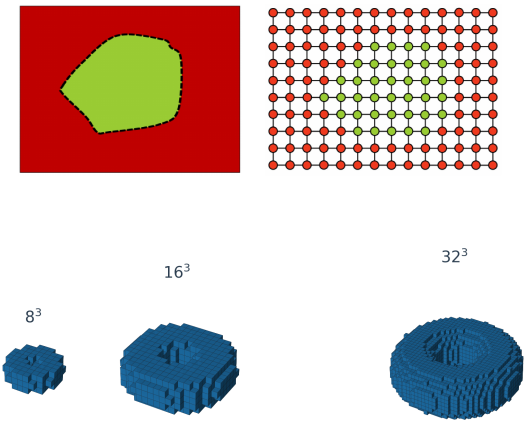
- **Binary-threshold** that grid: 1 inside, 0 outside
- A 3D **occupancy grid** — the direct analog of pixels



Thresholding figure after CS231n; tori generated

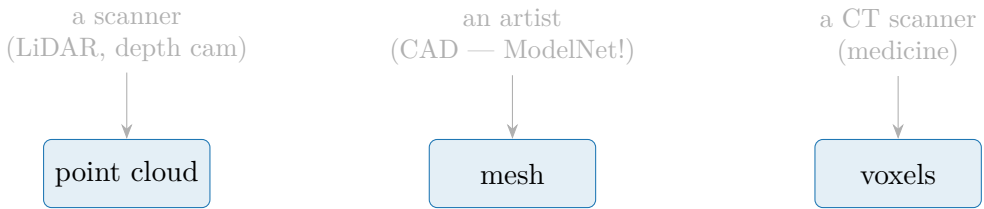
Voxels — Threshold the Grid

- **Binary-threshold** that grid: 1 inside, 0 outside
- A 3D **occupancy grid** — the direct analog of pixels
- Regular structure ⇒ convolutions work out of the box

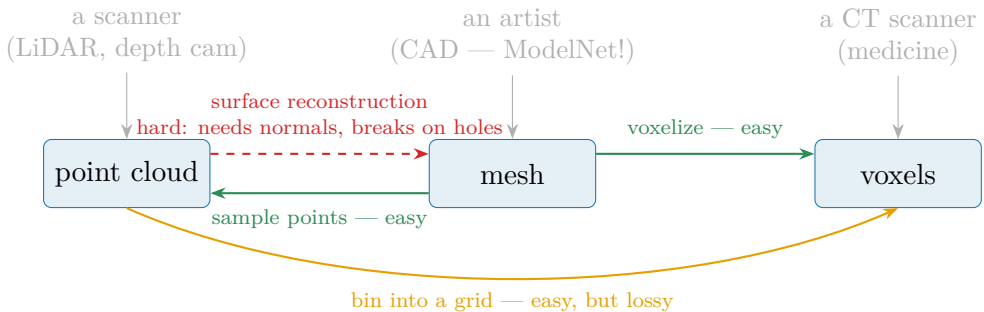


Thresholding figure after CS231n; tori generated

Where Does Each Format Come From?



Where Does Each Format Come From?



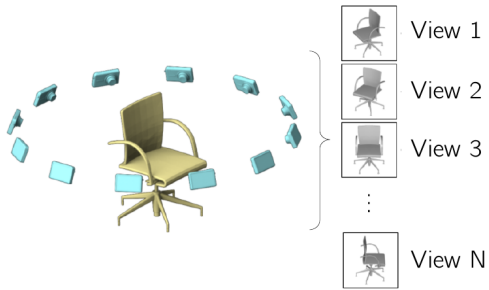
from a **mesh**, every conversion is cheap — from a **scan**, every conversion loses something.

Idea 1 — Render It: Multi-View CNN

the 3D shape we want to recognize

Su et al., ICCV 2015

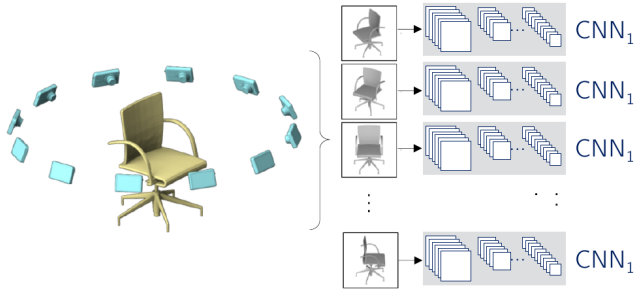
Idea 1 — Render It: Multi-View CNN



place N virtual cameras around it $\rightarrow$ render N ordinary images

Su et al., ICCV 2015

Idea 1 — Render It: Multi-View CNN

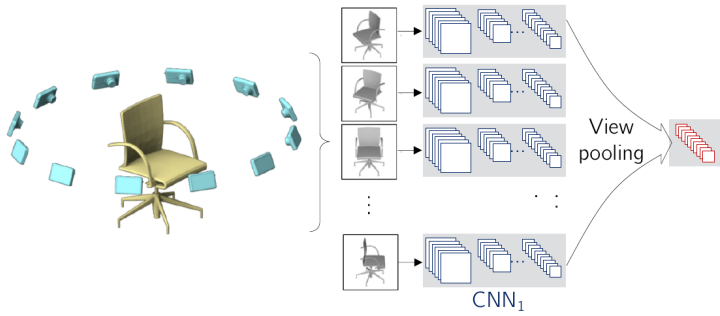


CNN_1: a ConvNet extracting image features

a **shared** ConvNet (ImageNet-pretrained) extracts per-view features

Su et al., ICCV 2015

Idea 1 — Render It: Multi-View CNN

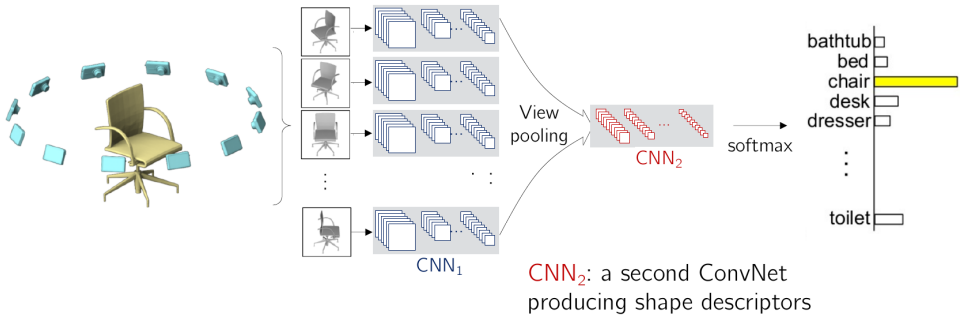


View pooling: element-wise max-pooling across all views

view pooling: element-wise **max** across views — view order stops mattering

Su et al., ICCV 2015

Idea 1 — Render It: Multi-View CNN



a second ConvNet turns the pooled descriptor into class scores

Su et al., ICCV 2015

MVCNN on ModelNet40

Method	Classification (acc.)	Retrieval (mAP)
SPH (non-deep descriptor)	68.2%	33.3%
LFD (non-deep descriptor)	75.5%	40.9%
3D ShapeNets (voxel net)	77.3%	49.2%
CNN on 12 views, no pooling	88.6%	62.8%
MVCNN, 12 views	**89.9%**	70.1%
MVCNN + metric, 12 views	89.5%	**80.2%**

Su et al., ICCV 2015; ModelNet40 benchmark

Multi-View: Pros and Cons

+ Excellent performance

+ Reuses the entire 2D toolbox: architectures, **pretrained weights**, augmentation

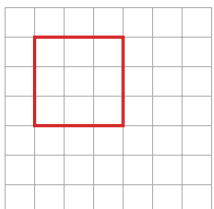
After Hao Su, CS231n

Multi-View: Pros and Cons

+ Excellent performance
+ Reuses the entire 2D toolbox: architectures, **pretrained weights**, augmentation
– Needs a **projection** step — geometry is decided before the network sees anything
– Real sensor data is a **noisy, incomplete point cloud** — partial scans render to broken images

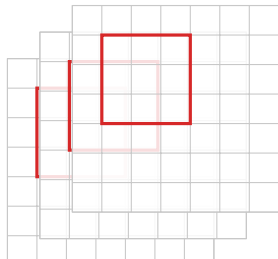
After Hao Su, CS231n

Idea 2 — Voxelize It: 3D Convolution



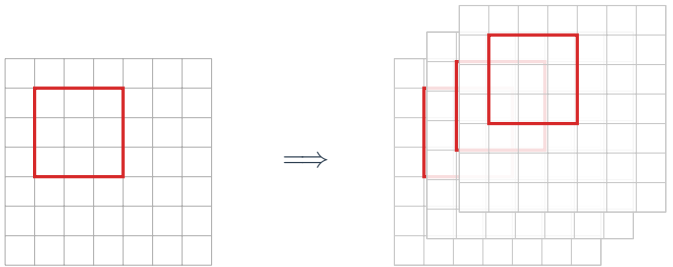
3×3 kernel slides in x, y

$\Longrightarrow$



$3 \times 3 \times 3$ kernel slides in x, y, z

Idea 2 — Voxelize It: 3D Convolution

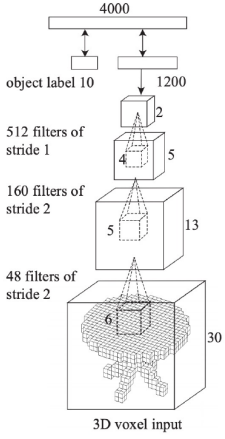


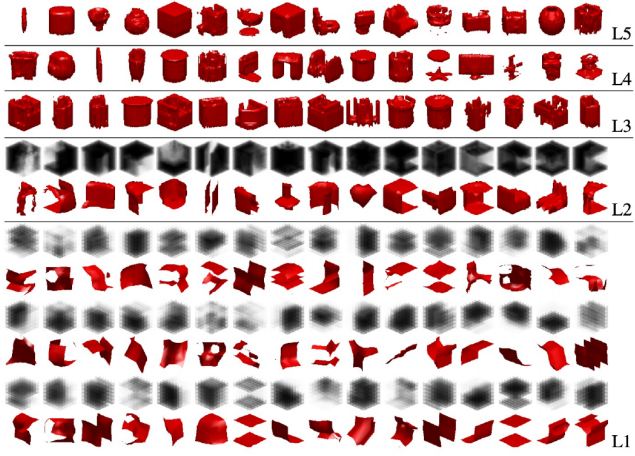
3×3 kernel slides in x, y $\qquad$ $3 \times 3 \times 3$ kernel slides in x, y, z

Same machinery, one more axis: $\quad \text{conv2d}(C{\times}H{\times}W) \;\rightarrow\; \text{conv3d}(C{\times}D{\times}H{\times}W)$

(`nn.Conv3d` in PyTorch)

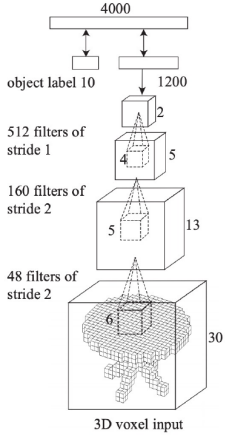
3D ShapeNets: the First Deep Net on Voxels

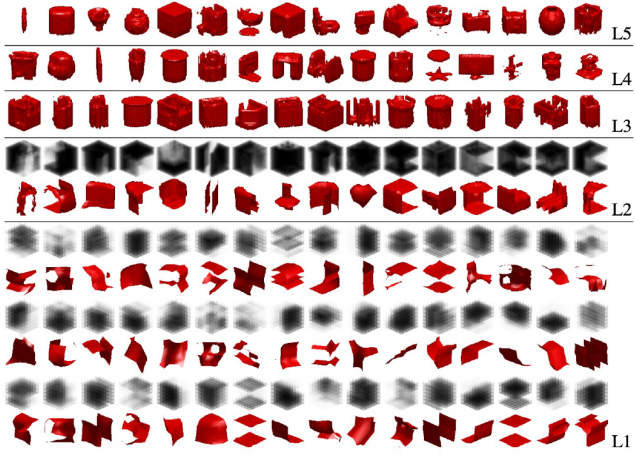




Wu et al., CVPR 2015

3D ShapeNets: the First Deep Net on Voxels





3D filters learn a hierarchy — surface patches at L1, object parts at L5 — just like 2D CNN features.

Wu et al., CVPR 2015

The Memory Wall

One feature map of a dense 3D CNN:

$$\underbrace{256^3}_{\text{voxels}} \times \underbrace{32}_{\text{channels}} \times \underbrace{4\,\text{B}}_{\text{float}} = \mathbf{2\ GB}$$

per layer, per sample — *before* gradients (× batch, × depth, × 2–3 for the backward pass)

The Memory Wall

One feature map of a dense 3D CNN:

$$\underbrace{256^3}_{\text{voxels}} \times \underbrace{32}_{\text{channels}} \times \underbrace{4\,\text{B}}_{\text{float}} = \mathbf{2\ GB}$$

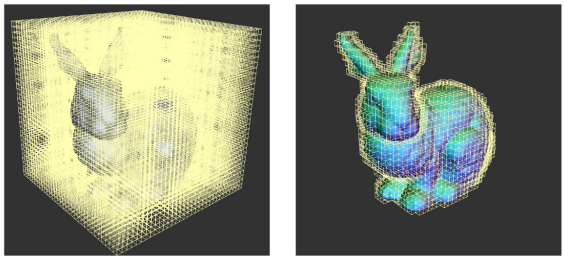
per layer, per sample — *before* gradients (× batch, × depth, × 2–3 for the backward pass)

Memory $\propto n^3$ — doubling the resolution costs $8\times$.

Dense voxel CNNs run out of memory around 64^3 — the fix is next.

Octrees: Spend Memory Only Near the Surface

- Shapes are **surfaces**: almost all voxels are empty or deep inside

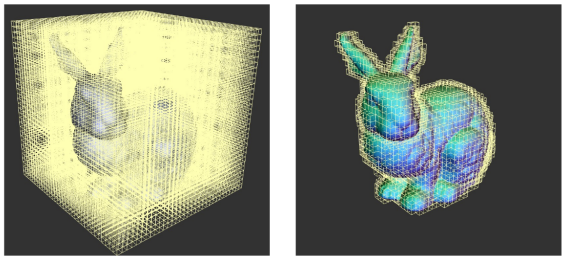


fine cells hug the surface; empty space stays coarse

Bunny figure: Hao Su, CS231n

Octrees: Spend Memory Only Near the Surface

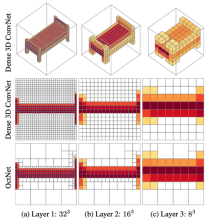
- Shapes are **surfaces**: almost all voxels are empty or deep inside
- Recursively subdivide **only cells that touch the surface**



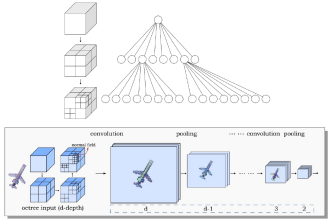
fine cells hug the surface; empty space stays coarse

Bunny figure: Hao Su, CS231n

Convolutions on Octrees



Riegler et al. OctNet. CVPR 2017

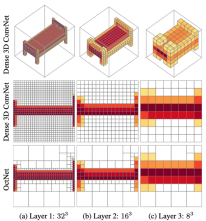


Wang et al. O-CNN. SIGGRAPH 2017

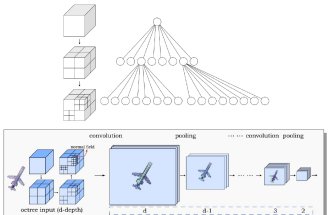
conv, pooling, features live **on the cells** —
never on empty space

Riegler et al., CVPR 2017 (OctNet); Wang et al., SIGGRAPH 2017 (O-CNN); memory replot qualitative

Convolutions on Octrees

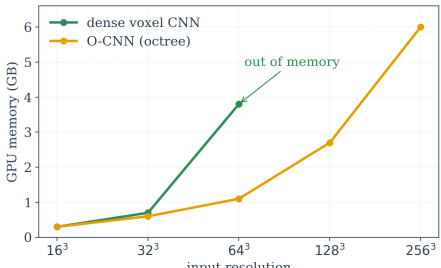


Riegler et al. OctNet. CVPR 2017



Wang et al. O-CNN. SIGGRAPH 2017

conv, pooling, features live **on the cells** — never on empty space

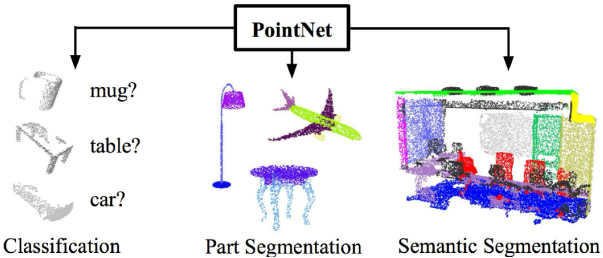


dense dies past 64^3 — the octree version trains at 256^3

Riegler et al., CVPR 2017 (OctNet); Wang et al., SIGGRAPH 2017 (O-CNN); memory replot qualitative

Idea 3 — Eat the Points: PointNet

- Skip rendering *and* voxelization: input is the raw $N \times 3$ point set
- **End-to-end learning** on irregular point data
- One backbone, many tasks: classification, part segmentation, scene parsing



Qi et al., CVPR 2017; figure: He Wang

What the Network Must Respect

- **Permutation invariance**
  - a point cloud is a **set**: all $N!$ orderings are the same shape
  - $f(x_1, \ldots, x_N)$ must not depend on the storage order

After He Wang, CS231n

What the Network Must Respect

- **Permutation invariance**
  - a point cloud is a **set**: all $N!$ orderings are the same shape
  - $f(x_1, \ldots, x_N)$ must not depend on the storage order
- **Sampling invariance**
  - the output should describe the **geometry**, not the **sampling**
  - denser or sparser scans of the same surface $\Rightarrow$ same answer

After He Wang, CS231n

What the Network Must Respect

- **Permutation invariance**
  - a point cloud is a **set**: all $N!$ orderings are the same shape
  - $f(x_1, \ldots, x_N)$ must not depend on the storage order
- **Sampling invariance**
  - the output should describe the **geometry**, not the **sampling**
  - denser or sparser scans of the same surface $\Rightarrow$ same answer

an MLP on the flattened $N \times 3$ vector violates both — swap two rows and the input changes.

After He Wang, CS231n

Permutation Invariance = Symmetric Functions

$$f(x_1, x_2, \ldots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \ldots, x_{\pi_n}) \qquad \forall \pi, \;\; x_i \in \mathbb{R}^D$$

After He Wang, CS231n

Permutation Invariance = Symmetric Functions

$$f(x_1, x_2, \ldots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \ldots, x_{\pi_n}) \qquad \forall \pi, \quad x_i \in \mathbb{R}^D$$

Examples:

$$f(x_1, \ldots, x_n) = \max\{x_1, \ldots, x_n\}$$
$$f(x_1, \ldots, x_n) = x_1 + x_2 + \cdots + x_n$$

After He Wang, CS231n

Permutation Invariance = Symmetric Functions

$$f(x_1, x_2, \ldots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \ldots, x_{\pi_n}) \qquad \forall \pi, \quad x_i \in \mathbb{R}^D$$

Examples:

$$f(x_1, \ldots, x_n) = \max\{x_1, \ldots, x_n\}$$
$$f(x_1, \ldots, x_n) = x_1 + x_2 + \cdots + x_n$$

Question: how do we build a *universal* family of symmetric functions with neural networks?

After He Wang, CS231n

Simplest Attempt: Aggregate the Raw Points

$(1, 2, 3)$

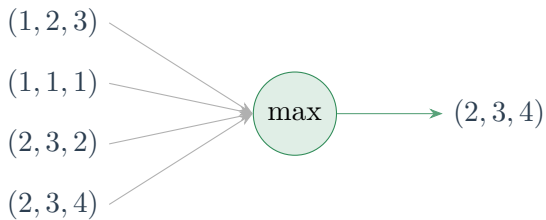
$(1, 1, 1)$

$(2, 3, 2)$

$(2, 3, 4)$

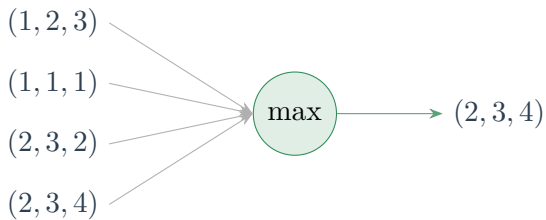
Example: He Wang, CS231n

Simplest Attempt: Aggregate the Raw Points



Example: He Wang, CS231n

Simplest Attempt: Aggregate the Raw Points



Symmetric, yes — but it only sees the **bounding extremes** of the shape.

Example: He Wang, CS231n

Vanilla PointNet: $\gamma \circ g \circ h$

$f(x_1, \ldots, x_n) = \gamma\big(g(h(x_1), \ldots, h(x_n))\big)$ is symmetric if g is symmetric

$(1, 2, 3)$

$(1, 1, 1)$

$(2, 3, 2)$

$\vdots$

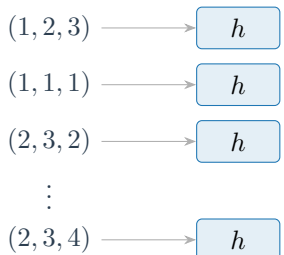
$(2, 3, 4)$

Qi et al., CVPR 2017; diagram after He Wang

Vanilla PointNet: $\gamma \circ g \circ h$

$f(x_1, \ldots, x_n) = \gamma\big(g(h(x_1), \ldots, h(x_n))\big)$ is symmetric if g is symmetric

shared MLP: $\mathbb{R}^3 \to \mathbb{R}^{1024}$

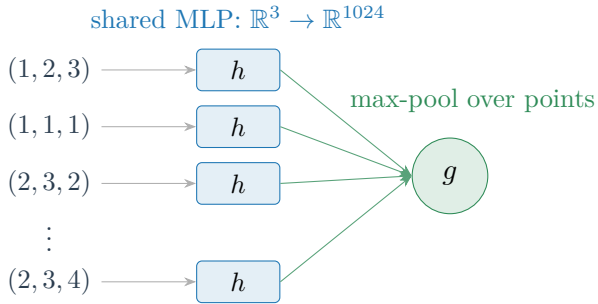


h lifts each point **independently** to a high-dimensional feature space

Qi et al., CVPR 2017; diagram after He Wang

Vanilla PointNet: $\gamma \circ g \circ h$

$f(x_1, \ldots, x_n) = \gamma\big(g(h(x_1), \ldots, h(x_n))\big)$ is symmetric if g is symmetric

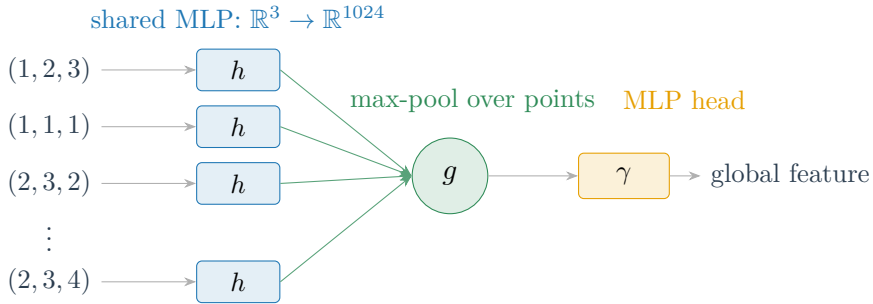


g = max-pool: symmetric, so f is permutation-invariant by construction

Qi et al., CVPR 2017; diagram after He Wang

Vanilla PointNet: $\gamma \circ g \circ h$

$f(x_1, \ldots, x_n) = \gamma\big(g(h(x_1), \ldots, h(x_n))\big)$ is symmetric if g is symmetric

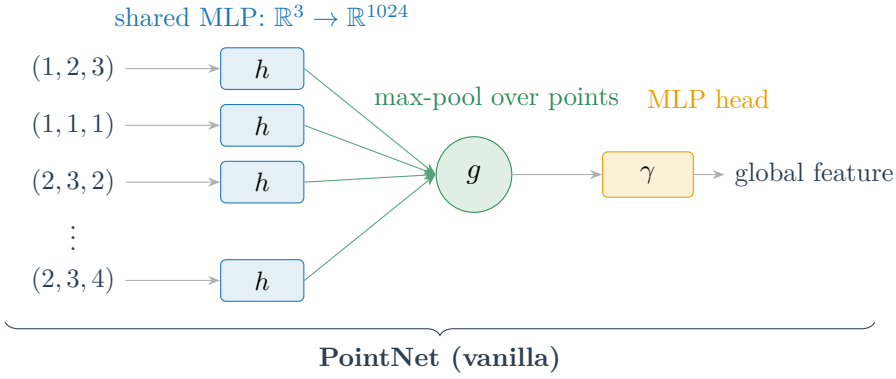


γ maps the pooled descriptor to the output — class scores, embeddings, …

Qi et al., CVPR 2017; diagram after He Wang

Vanilla PointNet: $\gamma \circ g \circ h$

$f(x_1, \ldots, x_n) = \gamma\big(g(h(x_1), \ldots, h(x_n))\big)$ is symmetric if g is symmetric

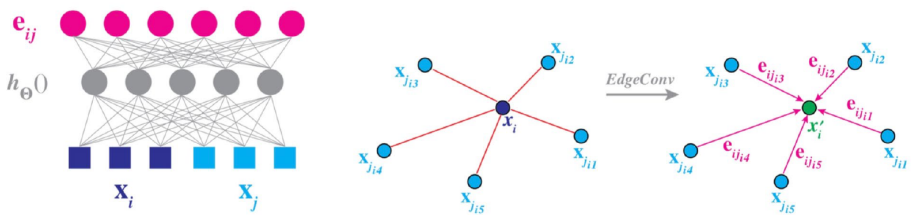


after the lift, **different points win different feature dimensions** — the max now preserves geometry, not just extremes

Qi et al., CVPR 2017; diagram after He Wang

Beyond PointNet: Graphs on Point Clouds

- PointNet treats every point **independently** until the final max
- Points → nodes, k-nearest neighbours → edges
- **EdgeConv**: learn on (point, neighbour) pairs ⇒ local structure returns



Wang et al., ACM TOG 2019 (DGCNN)

Comparing Point Clouds: Chamfer and EMD

$$d_{CD}(S_1, S_2) = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2$$

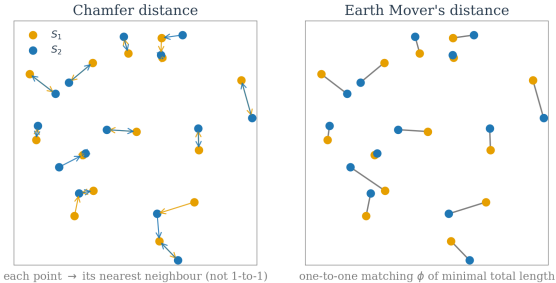
$$d_{EMD}(S_1, S_2) = \min_{\phi: S_1 \to S_2 \text{ bijective}} \sum_{x \in S_1} \|x - \phi(x)\|_2$$

Fan et al., CVPR 2017; after He Wang, CS231n

Comparing Point Clouds: Chamfer and EMD

$$d_{CD}(S_1, S_2) = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2$$

$$d_{EMD}(S_1, S_2) = \min_{\phi: S_1 \to S_2 \text{ bijective}} \sum_{x \in S_1} \|x - \phi(x)\|_2$$



Chamfer: nearest neighbour both ways (many-to-one allowed). EMD: cheapest **bijection** ϕ.

both **differentiable** ⇒ the training losses when a network outputs points — cross-entropy needs a grid.

Fan et al., CVPR 2017; after He Wang, CS231n

Summary

- 3D has **no canonical format** — points, meshes, voxels each trade off detail, memory, and structure
- Convolutions need **regularity**: render to images (multi-view) or voxelize to a grid to reuse them
- Dense voxels hit a **cubic memory wall**; octrees spend resolution only near the surface
- Sets need **symmetric functions**: PointNet = lift each point (h), max-pool (g), read out (γ)

References

- Su, Maji, Kalogerakis, Learned-Miller. *Multi-view CNNs for 3D Shape Recognition.* ICCV 2015.
- Wu, Song, Khosla, Yu, Zhang, Tang, Xiao. *3D ShapeNets: A Deep Representation for Volumetric Shapes.* CVPR 2015.
- Riegler, Ulusoy, Geiger. *OctNet: Learning Deep 3D Representations at High Resolutions.* CVPR 2017.
- Wang, Liu, Guo, Sun, Tong. *O-CNN: Octree-based Convolutional Neural Networks for 3D Shape Analysis.* SIGGRAPH 2017.
- Qi, Su, Mo, Guibas. *PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation.* CVPR 2017.
- Wang, Sun, Liu, Sarma, Bronstein, Solomon. *Dynamic Graph CNN for Learning on Point Clouds.* ACM TOG 2019.
- Fan, Su, Guibas. *A Point Set Generation Network for 3D Object Reconstruction from a Single Image.* CVPR 2017.

Slides based on Stanford CS231n (Spring 2025), Lecture 15: 3D Vision, Jiajun Wu; original slide credits: Hao Su, Ren Ng, He Wang.